Total	**4**	

Knowledge Units

GIT-Fundamentals: Fundamental Concepts

CS Core:

1. Uses of computer graphics and interactive techniques and their potential risks and abuses.
    a. Entertainment, business, and scientific applications: e.g., visual effects, generative imagery, computer vision, machine learning, user interfaces, video editing, games and game engines, computer-aided design and manufacturing, data visualization, and virtual/augmented/mixed reality
    b. Intellectual property, deep fakes, facial recognition, privacy (See also: SEP-DEIA, SEP-Privacy, SEP-IP, SEP-Professional-Ethics)
2. Graphic output
    a. Displays (e.g., LCD)
    b. Printers
    c. Analog film
    d. Concepts
        i. Resolution (e.g., pixels, dots)
        ii. Aspect ratio
        iii. Frame rate
3. Human vision system
    a. Tristimulus reception (RGB)
    b. Eye as a camera (projection)
    c. Persistence of vision (frame rate, motion blur)
    d. Contrast (detection, Mach banding, dithering/aliasing)
    e. Non-linear response (dynamic range, tone mapping)
    f. Binocular vision (stereo)
    g. Accessibility (color deficiency, strobing, monocular vision, etc.) (See also: SEP-DEIA, HCI-User)
4. Standard image formats
    a. Raster
        i. Lossless (e.g., TIF)
        ii. Lossy (e.g., JPG, GIF, etc.)
    b. Vector (e.g., SVG, Adobe Illustrator)
5. Digitization of analog data
    a. Rasterization
    b. Resolution
    c. Sampling and quantization
6. Color models: additive (RGB), subtractive (CMYK), and color perception (HSV)
7. Tradeoffs between storing image data and re-computing image data
8. Spatialization: coordinate systems, absolute and relative positioning
9. Animation as a sequence of still images

KA Core:

10. Applied interactive graphics (e.g., processing, python)
11. Display characteristics (protocols and ports)

Illustrative Learning Outcomes:

CS Core:

1. Identify common uses of digital presentation to humans (e.g., computer graphics, sound).
2. Describe how analog signals can be reasonably represented by discrete samples, for example, how images can be represented by pixels.
3. Compute the memory requirement for storing a color image given its resolution.
4. Create a graphic depicting how the limits of human perception affect choices about the digital representation of analog signals.
5. Indicate when and why you should use each of the following common file formats: JPG, PNG, MP3, MP4, and GIF.
6. Describe color models and their use in graphics display devices.
7. Compute the memory requirements for a multi-second movie (lasting *n* seconds) displaying at a specific framerate (*f* frames per second) at a specified resolution (*r* pixels per frame)
8. Compare and contrast digital video to analog video.
9. Describe the basic process of producing continuous motion from a sequence of discrete frames (sometimes called “flicker fusion”).
10. Describe a possible visual misrepresentation that could result from digitally sampling an analog world.
11. Compute memory space requirements based on resolution and color coding.
12. Compute time requirements based on refresh rates and rasterization techniques.

KA Core:

13. Design a user interface and an alternative for persons with color perception deficiency.
14. Construct a simple graphical user interface using a graphics library.

GIT-Visualization: Visualization

KA Core:

1. Scientific Data Visualization and Information Visualization
2. Visualization techniques
    a. Statistical visualization (e.g., scatterplots, bar graphs, histograms, line graphs, pie charts, trees, and graphs)
    b. Text visualization
    c. Geospatial visualization
    d. 2D/3D scalar fields
    e. Vector fields
    f. Direct volume rendering
3. Visualization pipeline
    a. Structuring data
    b. Mapping data to visual representations (e.g., scales, grammar of graphics)
    c. View transformations (e.g., pan, zoom, filter, select)

4. Common data formats (e.g., HDF, netCDF, geotiff, GeoJSON, shape files, raw binary, JSON, CSV, plain text)
5. High-dimensional data handling techniques
   a. Statistical (e.g., averaging, clustering, filtering)
   b. Perceptual (e.g., multi-dimensional vis, parallel coordinates, trellis plots)
6. Perceptual and cognitive foundations that drive visual abstractions.
   a. Human optical system
   b. Color theory
   c. Gestalt theories
7. Design and evaluation of visualizations
   a. Purpose (e.g., analysis, communication, aesthetics)
   b. Accessibility
   c. Appropriateness of encodings
   d. Misleading visualizations

Illustrative Learning Outcomes:

KA Core:

1. Compare and contrast data visualization and information visualization.
2. Deploy basic algorithms for visualization.
3. Compare the tradeoffs of visualization algorithms in terms of accuracy and performance.
4. Design a suitable visualization for a particular combination of data characteristics, application tasks, and audience.
5. Analyze the effectiveness of a given visualization for a particular task.
6. Design a process to evaluate the utility of a visualization algorithm or system.
7. Identify a variety of applications of visualization including representations of scientific, medical, and mathematical data; flow visualization; and spatial analysis.

GIT-Rendering: Applied Rendering and Techniques

KA Core: (See also: SPD-Game)

1. Object and scene modeling
   a. Object representations: polygonal, parametric, etc.
   b. Modeling transformations: affine and coordinate-system transformations
   c. Scene representations: scene graphs
2. Camera and projection modeling
   a. Pinhole cameras, similar triangles, and projection model
   b. Camera models
   c. Projective geometry
3. Radiometry and light models
   a. Radiometry
   b. Rendering equation
   c. Rendering in nature – emission and scattering, etc.
4. Rendering
   a. Simple triangle rasterization
   b. Rendering with a shader-based API

c. Visibility and occlusion, including solutions to this problem (e.g., depth buffering, Painter's algorithm, and ray tracing)
d. Texture mapping, including minification and magnification (e.g., trilinear MIP mapping)
e. Application of spatial data structures to rendering.
f. Ray tracing
g. Sampling and anti-aliasing

Illustrative Learning Outcomes:

KA Core:

1. Describe and illustrate the light transport problem (i.e., light is emitted, scatters around the scene, and is measured by the eye).
2. Describe the basic rendering pipeline.
3. Compare and contrast how forward and backwards rendering factor into the graphics pipeline.
4. Create a program to display 2D shapes in a window.
5. Create a program to display 3D models.
6. Produce linear perspective from similar triangles by converting points (x, y, z) to points (x/z, y/z, 1).
7. Compute two-dimensional and 3-dimensional points by applying affine transformations.
8. Indicate the changes required to extend 2D transformation operations to handle transformations in 3D.
9. Define texture mapping, sampling, and anti-aliasing, and describe examples of each.
10. Compare ray tracing and rasterization for the visibility problem.
11. Construct a program that performs transformation and clipping operations on simple two-dimensional shapes.
12. Implement a simple real-time renderer using a rasterization API (e.g., OpenGL, webGL) using vertex buffers and shaders.
13. Compare and contrast the different rendering techniques.
14. Compare and contrast the difference in transforming the camera vs the models.

GIT-Modeling: Geometric Modeling

KA Core:

1. Basic geometric operations such as intersection calculation and proximity tests on 2D objects
2. Surface representation/model
    a. Tessellation
    b. Mesh representation, mesh fairing, and mesh generation techniques such as Delaunay triangulation, and marching cubes/tetrahedrons
    c. Parametric polynomial curves and surfaces
    d. Implicit representation of curves and surfaces
    e. Spatial subdivision techniques
3. Volumetric representation/model
    a. Volumes, voxels, and point-based representations.
    b. Signed Distance Fields
    c. Sparse Volumes, i.e., VDB
    d. Constructive Solid Geometry (CSG) representation
4. Procedural representation/model

   a. Fractals
   b. L-Systems
5. Multi-resolution modeling (See also: SPD-Game)
6. Reconstruction, e.g., 3D scanning, photogrammetry

Illustrative Learning Outcomes:
KA Core:
1. Contrast representing curves and surfaces in both implicit and parametric forms.
2. Create simple polyhedral models by surface tessellation.
3. Create a mesh representation from an implicit surface.
4. Create a fractal model or terrain using a procedural method.
5. Create a mesh from data points acquired with a laser scanner.
6. Create CSG models from simple primitives, such as cubes and quadric surfaces.
7. Contrast modeling approaches with respect to space and time complexity and quality of image.

GIT-Shading: Shading and Advanced Rendering

KA Core:
1. Solutions and approximations to the rendering equation, for example
   a. Distribution ray tracing and path tracing
   b. Photon mapping
   c. Bidirectional path tracing
   d. Metropolis light transport
2. Time (motion blur), lens position (focus), and continuous frequency (color) and their impact on rendering
3. Shadow mapping
4. Occlusion culling
5. Bidirectional Scattering Distribution function (BSDF) theory and microfacets
6. Subsurface scattering
7. Area light sources
8. Hierarchical depth buffering
9. Image-based rendering
10. Non-photorealistic rendering
11. Realtime rendering
12. GPU architecture (See also: AR-Heterogeneity)
13. Human visual systems including adaptation to light, sensitivity to noise, and flicker fusion (See also: HCI-Accessibility, SEP-DEIA)

Illustrative Learning Outcomes:
KA Core:
1. Describe how an algorithm estimates a solution to the rendering equation.
2. Discuss the properties of a rendering algorithm (e.g., complete, consistent, and unbiased).
3. Analyze the bandwidth and computation demands of a simple shading algorithm.
4. Implement a non-trivial shading algorithm (e.g., toon shading, cascaded shadow maps) under a rasterization API.

5. State how a particular artistic technique might be implemented in a renderer.
6. Describe how one might recognize the shading techniques used to create a particular image.
7. Write a program that implements any of the specified graphics techniques using a primitive graphics system at the individual pixel level.
8. Write a ray tracer for scenes using a simple (e.g., Phong's) Bidirectional Reflection Distribution Function (BRDF) plus reflection and refraction.

GIT-Animation: Computer Animation

KA Core:

1. Principles of Animation: Squash and Stretch, Timing, Anticipation, Staging, Follow Through and Overlapping Action, Straight Ahead Action, and Pose-to-Pose Action, Slow In and Out, Arcs, Exaggeration, and Appeal
2. Types of animation
    a. 2- and 3-dimensional animation
    b. Motion graphics
    c. Motion capture
    d. Motion graphics
    e. Stop animation
3. Key-frame animation
    a. Keyframe Interpolation Methods: Lerp/Slerp/Spline
4. Forward and inverse kinematics (See also: SPD-Robot, AI-Robotics)
5. Skinning algorithms
    a. Capturing
    b. Linear blend, dual quaternion
    c. Rigging
    d. Blend shapes
    e. Pose space deformation
6. Motion capture
    a. Set up and fundamentals
    b. Blending motion capture clips
    c. Blending motion capture and keyframe animation
    d. Ethical considerations (See also: SEP-DEIA, SEP-Privacy)
        i. Avoidance of "default" captures - there is no typical human walk cycle.
        ii. Accessibility

Illustrative Learning Outcomes:

KA Core:

1. Using a simple open-source character model and rig, describe visually why each of the principles of animation is fundamental to realistic animation.
2. Compute the location and orientation of model parts using a forward kinematic approach.
3. Compute the orientation of articulated parts of a model from a location and orientation using an inverse kinematic approach.
4. Compare the tradeoffs in different representations of rotations.

5. Write a script that implements the spline interpolation method for producing in-between positions and orientations.
6. Deploy off-the-shelf animation software to construct, rig, and animate simple organic forms.

GIT-Simulation: Simulation

KA Core:

1. Collision detection and response
    a. Signed Distance Fields
    b. Sphere/sphere
    c. Triangle/point
    d. Edge/edge
2. Procedural animation using noise
3. Particle systems
    a. Integration methods (e.g., forward Euler, midpoint, leapfrog)
    b. Mass/spring networks
    c. Position-based dynamics
    d. Rules (e.g., boids, crowds)
    e. Rigid bodies
4. Grid-based fluids
    a. Semi-Lagrangian advection
    b. Pressure projection
5. Heightfields
    a. Terrain: transport, erosion
    b. Water: ripple, shallow water.
6. Rule-based systems (e.g., L-systems, space-colonizing systems, Game of Life)

Illustrative Learning Outcomes:

KA Core:

1. Implement algorithms for physical modeling of particle dynamics using simple Newtonian mechanics (e.g., Witkin & Kass, snakes and worms, symplectic Euler, Stormer/Verlet, or midpoint Euler methods)
2. Contrast the basic ideas behind fluid simulation methods for modeling ballistic trajectories (e.g., for splashes, dust, fire, or smoke).
3. Implement a smoke solver with user interaction.

GIT-Immersion: Immersion

KA Core: (See also: SPD-Game, SPD-Mobile, HCI-Design)

1. Immersion levels (i.e., Virtual Reality (VR), Augmented Reality (AR), and Mixed Reality (MR))
2. Definitions of and distinctions between immersion and presence
3. 360 Video
4. Stereoscopic display
    a. Head-mounted displays
    b. Stereo glasses

5. Viewer tracking
    a. Inside out and outside In
    b. Head/Body/Hand/tracking
6. Time-critical rendering to achieve optimal Motion To Photon (MTP) latency
    a. Multiple Levels Of Details (LOD)
    b. Image-based VR
    c. Branching movies
7. Distributed VR, collaboration over computer network
8. Presence and factors that impact level of immersion
9. 3D interaction
10. Applications in medicine, simulation, training, and visualization
11. Safety in immersive applications
    a. Motion sickness
    b. VR obscures the real world, which increases the potential for falls and physical accidents

Illustrative Learning Outcomes:

KA Core:

1. Create a stereoscopic image.
2. Design and write an AR or VR application.
3. Summarize the pros and cons of different types of viewer tracking.
4. Compare and contrast the differences between geometry- and image-based virtual reality.
5. Analyze the design issues of user action synchronization and data consistency in a networked environment.
6. Create the specifications for an augmented reality application to be used by surgeons in the operating room.
7. Assess an immersive application's accessibility (See also: HCI-Accessibility, SEP-DEIA)
8. Identify the most important technical characteristics of a VR system/application that should be controlled to avoid motion sickness and explain why.

GIT-Interaction: Interaction

KA Core:

1. Event Driven Programming (See also: FPL-Event-Driven)
    a. Mouse or touch events
    b. Keyboard events
    c. Voice input
    d. Sensors
    e. Message passing communication
    f. Network events
2. Graphical User Interface (Single Channel)
    a. Window
    b. Icons
    c. Menus
    d. Pointing Devices
3. Accessibility (See also: SEP-DEIA)

Non-core:

4. Gestural Interfaces (See also: SPD-Game)
    a. Touch screen gestures
    b. Hand and body gestures
5. Haptic Interfaces
    a. External actuators
    b. Gloves
    c. Exoskeletons
6. Multimodal Interfaces
7. Head-worn Interfaces
    a. Brain-computer interfaces, e.g., Electroencephalography (EEG) electrodes and Multi-Electrode Arrays (MEAs)
    b. Headsets with embedded eye tracking
    c. AR glasses
8. Natural Language Interfaces (See also: AI-NLP)

Illustrative Learning Outcomes:

KA Core:

1. Create a simple game that responds to single channel mouse and keyboard events.
2. Create a mobile app that responds to touch events.
3. Design and create an application that responds to different event triggers.

None-core:

4. Assess the consistency or lack of consistency in cross-platform touch screen gestures.
5. Design and create an application that provides haptic feedback.
6. Write a program that is controlled by gestures.

GIT-Image: Image Processing

KA Core: (See also: AI-Vision)

1. Morphological operations
    a. Connected components
    b. Dilation
    c. Erosion
    d. Computing region properties (area, perimeter, centroid, etc.)
2. Color histograms
    a. Representation
    b. Contrast enhancement through normalization
3. Image enhancement
    a. Convolution
    b. Blur (e.g., Gaussian)
    c. Sharpen (e.g., Laplacian)
    d. Frequency filtering (e.g., low-pass, high-pass)
4. Image restoration

   a. Noise, degradation
   b. Inpainting and other completion algorithms
   c. Wiener filter
5. Image coding
   a. Redundancy
   b. Compression (e.g., Huffman coding)
   c. Discrete Cosine Transform (DCT), wavelet transform, Fourier transforms (See also: SPD-Interactive)
   d. Nyquist Theorem
   e. Watermarks
6. Connections to deep learning (e.g., Convolutional Neural Networks) (See also: AI-ML)

Illustrative Learning Outcomes:

KA Core:

1. Write a program that uses dilation and erosion to smooth the edges of a binary image.
2. Manipulating the hue of an image.
3. Write a program that applies a high-pass filter to an image. (The advanced variation would be to filter an image using a high-pass filter in the frequency domain.)
4. Write a program that restores missing parts of an image using an in-paint algorithm (e.g., Poisson image editing)
5. Assess the results of selectively filtering an image in the frequency domain.

GIT-Physical: Tangible/Physical Computing

KA Core:

1. Interaction with the physical world (See also: SPD-Embedded)
   a. Acquisition of data from sensors
   b. Driving external actuators
2. Connection to physical artifacts
   a. Computer-Aided Design (CAD)
   b. Computer-Aided Manufacturing (CAM)
   c. Fabrication (See also: HCI-Design)
      i. Prototyping
      ii. Additive (3D printing)
      iii. Subtractive (Computer Numerical Control (CNC) milling)
      iv. Forming (vacuum forming)
3. Internet of Things (See also: SPD-Interactive)
   a. Network connectivity
   b. Wireless communication

Illustrative Learning Outcomes:

KA Core:

1. Construct a simple virtual switch or application button and use it to turn on an LED.
2. Construct a simple system to move a servo in response to sensor data.

3. Create a circuit and accompanying microcontroller code that uses a light sensor to vary a property of something else (e.g., color or brightness of an LED or graphic, position of an external actuator).
4. Create a circuit with a variable resistor and write a microcontroller program that reads and responds to the resistor's changing values.
5. Create a 3D form in a CAD package.
    a. Show how affine transformations are achieved in the CAD program.
    b. Show an example of instances of an object.
    c. Create a fabrication plan. Provide a cost estimate for materials and time. How will you fabricate it?
    d. Fabricate it. How closely did your actual fabrication process match your plan? Where did it differ?
6. Write the G- and M-Code to construct a 3D maze and use a CAD/CAM package to check your work.
7. Decide and defend your decision to use Ethernet, WiFi, Bluetooth, RFID/NFC, or something else for internet connectivity when designing an IoT pill dispenser. Create an IoT pill dispenser.
8. Distinguish between the different types of fabrication and describe when you would use each.

GIT-SEP: Society, Ethics, and the Profession

KA Core:

1. Accessibility in immersive, interactive, and physical computing applications (See also: SEP-DEIA)
    a. Accessible to people with mobility impairments
    b. Accessible to people with vision and/or hearing impairments
2. Ethics/privacy in graphics applications. (See also: SEP-Privacy, SEP-Professional-Ethics, and SEP-Security)
    a. Acquisition of private data (room scans, body proportions, active cameras, etc.)
    b. Can't look away from immersive applications easily
    c. Danger to self/surroundings while immersed
    d. Ethical pitfalls of facial recognition
    e. Misleading visualizations
        i. Due to incorrect data because of exaggeration, hole filling, smoothing, data cleanup, etc.
        ii. Even correct data can mislead (e.g., aliasing can cause back moving or stopped fan blades)
    f. Privacy regarding health and other personal information
    g. Bias in image processing
        i. Deep fakes
        ii. Applications that misidentify people based on skin color or hairstyle
3. Intellectual Property law as it relates to computer graphics and interactive techniques (See also: SEP-IP)
    a. images used to train generative AI
    b. images produced by generative AI
4. Current and past contributors to the field (See also: SEP-DEIA)

Illustrative Learning Outcomes:

KA Core:

1. Discuss the security issues inherent in location tags.
2. Describe the ethical pitfalls of facial recognition. Can facial recognition be used ethically? If so, how?
3. Discuss the copyright issues of using watermarked images to train a neural network.

Professional Dispositions

- **Self-directed:** Graphics hardware and software evolves rapidly. Students need to understand the importance of being a life-long learner.
- **Collaborative:** Graphics developers typically work in diverse teams composed of people with disparate subject matter expertise. Students should understand the value of being a good team member, and their teamwork skills should be cultivated and evaluated with constructive feedback.
- **Effective communicator:** Communication is critical. Students' technical communication—verbal, written, and in code—should be practiced and evaluated.
- **Creative:** Creative problem-solving lies at the core of computer graphics.

Mathematics Requirements

Required:

1. Coordinate geometry
2. Trigonometry
3. MSF-Linear*
   a. Points (coordinate systems & homogeneous coordinates), vectors, and matrices
   b. Vector operations: addition, scaling, dot and cross products
   c. Matrix operations: addition, multiplication, determinants
   d. Affine transformations
4. MSF-Calculus*
   a. Continuity

*Note, if students enroll in a graphics class without linear algebra or calculus, graphics faculty can teach what is needed. To wit, many graphics textbooks cover the requisite mathematics in the appendix.

Desirable:

1. MSF-Linear
   a. Eigenvectors and Eigen decomposition
   b. Gaussian elimination and lower upper factorization
   c. Singular value decomposition
2. MSF-Calculus
   a. Quaternions
   b. Differentiation
   c. Vector calculus
   d. Tensors
   e. Differential geometry
3. MSF-Probability